

Agenda

- Introduction
- Android Architecture
- Installation
- Creating First Android Application
- File Structure of Project
- Gradle
- Fundamental Components
- Activity Life cycle

Language, Libraries, Framework and Technology

1. Language

- It is a set of instructions/commands that are designed to communicate with computer(OS)
- it has its own syntax, grammar, rules, etc

2. Libraries

- Using the language we can define multiple functionalities.
- We can group all such functionalities to create our own library

3. Framework

- It is a structure on which you can build software
- It serves as a foundation, so you're not starting entirely from scratch
- A framework is a supporting structure that requires you to follow the pattern for application development

4. Technology

- Technology is the way in which a current problem statement is Resolved.
- It is a term used to define the solution to our problem statement
- It is mixture of Language, Libraries and framework

Feature vs Smart

- Any device that is designed to do minimal operations is called as feature device
- eg-> remote, mobilephones, washing machine etc.
- Any device that has got its own Operating system is called as smart device
- eg-> smartphones, smartTV, smartWatch, smartRefrigerators

History

- Android is an operating system that is designed on top of Linux OS with some changes
- It is designed for mobile devices
- It is free and open source OS
- It is developed by Andy Rubin in 2003
- It was acquired by Google in 2005

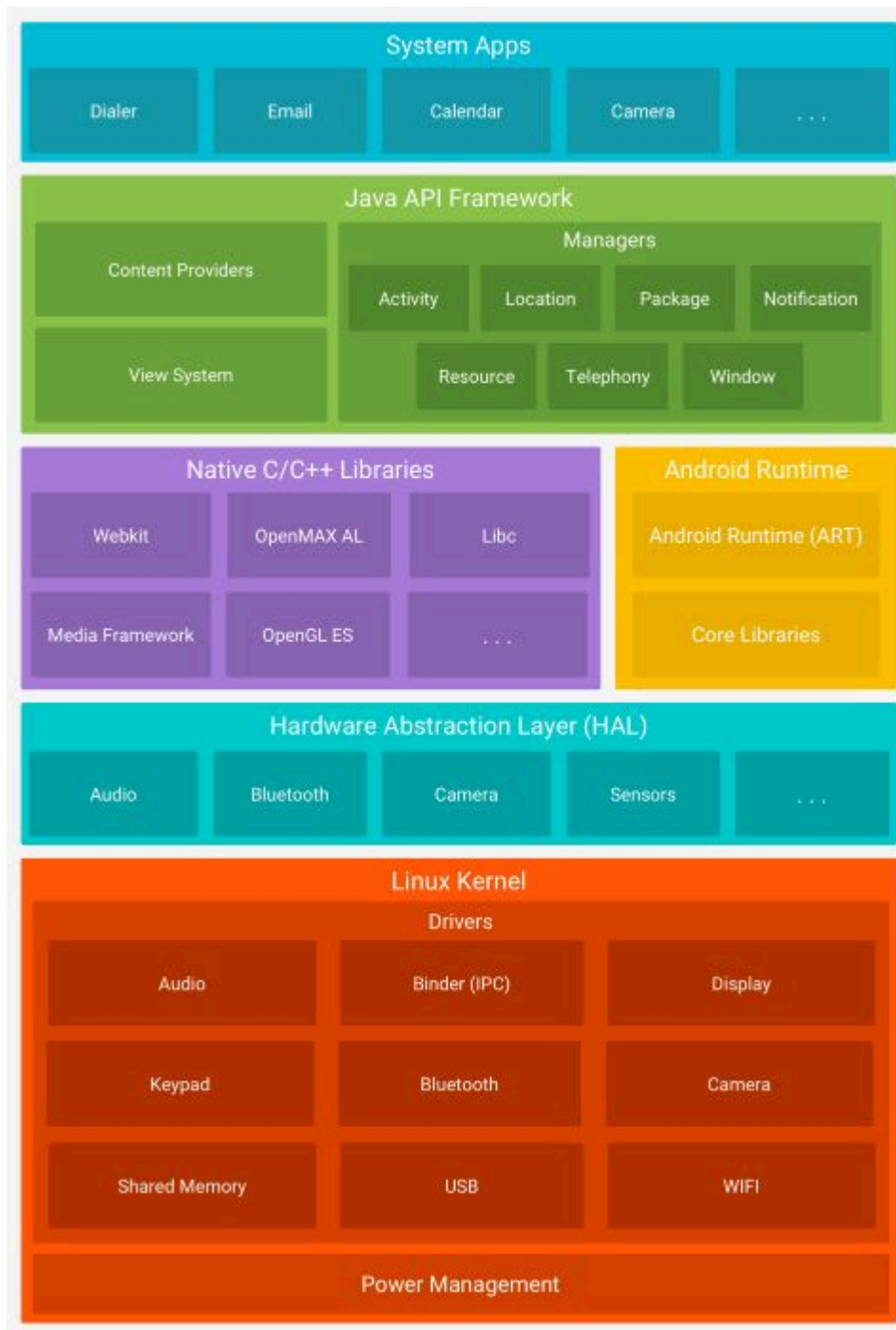
- In 2007, google announced the development of Andorid OS
- In 2008, HTC launced its first Android device

Versions of Android

- 1.0 -> Android 1.0(September 2008)
- 1.1 -> Android 1.1 (Feburary 2009)
- 1.5 -> Android Cupcake (April 2009)
- 1.6 -> Android Donut (September 2009)
- 2.0 -> Android Eclair - 2.0 (October 2009) - 2.0.1 (December 2009) - 2.1 (January 2010)
- 2.2 & 2.2.3 -> Android Froyo (May 2010)
- 2.3 -> Android GingerBread - 2.3 - 2.3.2 (December 2010) - 2.3.3 - 2.3.7 (Feburary 2011)
- 3.0 -> Android Honeycomb - 3.0 (Feburary 2011) - 3.1 (May 2011) - 3.2-3.2.6 (July 2011)
- 4.0 -> Android Ice Cream Sandwich - 4.0 - 4.0.2 (October 2011) - 4.0.3 - 4.0.4 (December 2011)
- 4.1 -> Android Jelly Bean - 4.1 – 4.1.2 (July 2012) - 4.2 – 4.2.2 (November 2012) - 4.3 – 4.3.1 (July 2013)
- 4.2 -> Android KitKat - 4.4 – 4.4.4 (October 2013) - 4.4W – 4.4W.2 (June 2014)
- 5.0 -> Android Lollipop - 5.0 – 5.0.2 (November 2014) - 5.1 – 5.1.1 (March 2015)
- 6.0 & 6.0.1 -> Android Marshmallow (September 2015)
- 7.0 -> Android Nougat - 7.0 (August 2016) - 7.1 - 7.1.2 (October 2016)
- 8.0 -> Android Oreo - 8.0 (August 2017) - 8.1 (December 2017)
- 9.0 -> Android Pie (August 2018)
- 10 -> Android 10/ Android Q / Quince Tart (Sepetember 2019)
- 11 -> Android 11 / Red Velvet Cake (Sepetember 2020)
- 12 -> Android 12 - 12 (October 2021) / Snow Cone - 12.1 (March 2022) / Snow Cone V2
- 13 -> Android 13 / Tiramisu (August 2022)
- 14 -> Android 14 / Upside Down Cake (October 2023)
- 15 -> Android 15/ Vanila IceCream (September 2024)
- 16 -> Android 16 / Baklava (June 2025)
- 17 -> Android 17 / Cinnamon Bun (Feburary 2026)

Android Architecture

- Android is an open source, Linux-based software stack created for a wide array of devices and form factors.
- Diagram shows the major components of the Android platform.



1. Applications

- It is the top layer of android architecture.
- The pre-installed applications like home, contacts, camera, gallery etc and third party applications downloaded from the play store like chat applications, games etc. will be installed on this layer.

2. Application Framework

- Application Framework provides several important classes which are used to create an Android application.
- It includes different types of services like activity manager, notification manager, view system, package manager etc. which are helpful for the development of our application

3. Android Runtime

- It is JVM for android, faster than desktop jvm.
- It uses 30% native instructions and 70% byte codes
- It is possible because of double compilation that android uses.
- The byte code is given to dex compiler which compiles and produces, 70% byte code and 30% native code in .dex(dalvik executables) file.
- For devices running Android version 5.0 (API level 21) or higher, each app runs in its own process and with its own instance of the Android Runtime (ART). - ART is written to run multiple virtual machines on low-memory devices by executing Dalvik Executable format (DEX) files, a bytecode format designed specifically for Android that's optimized for a minimal memory footprint.
- Some of the major features of ART include the following:
 - Ahead-of-time (AOT) and just-in-time (JIT) compilation
 - Optimized garbage collection (GC)
 - On Android 9 (API level 28) and higher, conversion of an app package's DEX files to more compact machine code
- Prior to Android version 5.0 (API level 21), Dalvik was the Android runtime. If your app runs well on ART, then it can work on Dalvik as well, but the reverse might not be true.
- Android also includes a set of core runtime libraries that provide most of the functionality of the Java programming language, including some Java 8 language features, that the Java API framework uses.

4. Platform Libraries

- The Platform Libraries includes various C/C++ core libraries and Java based libraries such as Media, Graphics, Surface Manager, OpenGL etc. to provide a support for android development.
- Android has its own C runtime library.
- The name for the Android's runtime library is Bionic.

5. Hardware abstraction layer (HAL)

- The hardware abstraction layer (HAL) provides standard interfaces that expose device hardware capabilities to the higher-level Java API framework.
- The HAL consists of multiple library modules, each of which implements an interface for a specific type of hardware component, such as the camera or Bluetooth module.
- When a framework API makes a call to access device hardware, the Android system loads the library module for that hardware component.

6. Linux Kernel

- Linux Kernel is heart of the android architecture.
- For android operating system the linux kernel is customized.
- Changes added to android from main line kernel is called as Androdism.
- It manages all the available drivers such as display drivers, camera drivers, Bluetooth drivers, audio drivers, memory drivers, etc. which are required during the runtime.
- The Linux Kernel will provide an abstraction layer between the device hardware and the other components of android architecture.
- The features of Linux kernel are:
 1. Security: The Linux kernel handles the security between the application and the system.
 2. Memory Management: It efficiently handles the memory management thereby providing the freedom to develop our apps.

3. Process Management: It manages the process well, allocates resources to processes whenever they need them.
4. Network Stack: It effectively handles the network communication.
5. Driver Model: It ensures that the application works properly on the device and hardware manufacturers responsible for building their drivers into the Linux build.

Installation

- Installation of Android Studio and SDK
- For mobile devices
 - Check which android version you are using
 - Update the device to the latest Security Patch
 - Turn on developers option
 - depending on the device(samsung,moto,nokia,realme,xiomi,...) the option will be available in the settings
 - Check the cable it must support the file transfer

Creating First Android Application

- click on create new project
- select for which device you want to create a project (Phone or Tablet)
- select EmptyViewsActivity and click on next
- Use proper name of the project
- use proper name for the package
- select the directory/workspace where you want to store the project
- select the language as java.
- select the proper min sdk for your application.
- click on finish
- wait till the gradle finishes its task.

File Structure

1. manifest Folder

- Manifests folder contains AndroidManifest.xml of our android application.
- This file contains information about our application such as the Android version, metadata and other application components.
- It acts as an intermediary between android OS and our application.

2. java Folder

- The Java folder contains all the java source code (.java) files that we create during the app development,including other Test files.

3. res (Resource Folder)

- The resource folder is the most important folder because it contains all the non-code sources like images,XML layouts, and UI strings for our android application.
- the sub folders inside this res folder are

1. Drawable
 - It is used to store the images/logos used in our application
2. Layout
 - It is used to create the UI of our application
3. mipmap
 - It is used to have icon for our application
4. values
 - It is used to keep constants that we can use across our application
5. anim
 - It is used to keep the animation xml files
6. menu
 - It is used to keep the menu xml files for the menus that we create

4. Gradle Scripts

1. In build.gradle (Project) there are buildscripts
2. In build.gradle (Module) plugins and implementations are used to build configurations that can be applied to all our application modules.

Gradle

- It is a build automation tool
- It is a open source tool
- It is used to build the softwares that we design
- It can build any type of softwares
- Android studio uses this gradle for the automated build system for its application that we design

Fundamental Components of Android

- The fundamental components are also called as Pillars of Android
- Their are 4 fundamental components for Android application

1. Activity

- An activity is the entry point for interacting with the user.
- It represents a single screen with a user interface.

2. Services

- A service is a general-purpose entry point for keeping an app running in the background for all kinds of reasons.
- It is a component that runs in the background to perform long-running operations or to perform work for remote processes.
- A service does not provide a user interface.

3. Content Provider

- A content provider manages a shared set of app data that you can store in the file system, in a SQLite database or on any other persistent storage location that your app can access.
- Through the content provider, other apps can query or modify the data, if the content provider permits it.

4. BroadCast Receiver

- A broadcast receiver is a component that lets the system deliver events to the app outside of a regular user flow so the app can respond to system-wide broadcast announcements.
- Because broadcast receivers are another well-defined entry into the app, the system can deliver broadcasts even to apps that aren't currently running.

Activity

- An activity is the entry point of an app for interacting with the user.
- It represents a single screen with a user interface.
- For example, an email app might have one activity that shows a list of new emails.
- another activity to compose an email, and another activity for reading emails.
- Although the activities work together to form a better user experience in the email app
- An activity goes through a number of states over the course of its lifetime
- a series of callbacks is used to handle transitions between states.
- the callbacks are as follows :

1. onCreate()

- You must implement this callback, which fires when the system creates your activity.
- You should initialize the essential components of your activity inside this callback
- For example, your app should create views and bind data to lists here.
- Most importantly, this is where you must call `setContentView()` to define the layout for the activity's user interface.
- When `onCreate()` finishes, the next callback is always `onStart()`.

2. onStart()

- As `onCreate()` exits, the activity enters the Started state, and the activity becomes visible to the user.
- This callback contains what amounts to the activity's final preparations for coming to the foreground and becoming interactive.
- When `onStart()` finishes, the next callback is always `onResume()`

3. onResume()

- At this point, the activity is at the top of the activity stack
- It captures all user input.
- The `onPause()` callback always follows `onResume()`.

4. onPause()

- The system calls `onPause()` when the activity loses focus and enters a Paused state. - This state occurs when, for example, the user taps the Back or Recents button.
- When the system calls `onPause()` for your activity, it technically means your activity is still partially visible, but most often is an indication that the user is leaving the activity, and the activity will soon enter the Stopped or Resumed state.
- Once `onPause()` finishes executing the next callback is either `onStop()` or `onResume()`, depending on what happens after the activity enters the Paused state.

5. onStop()

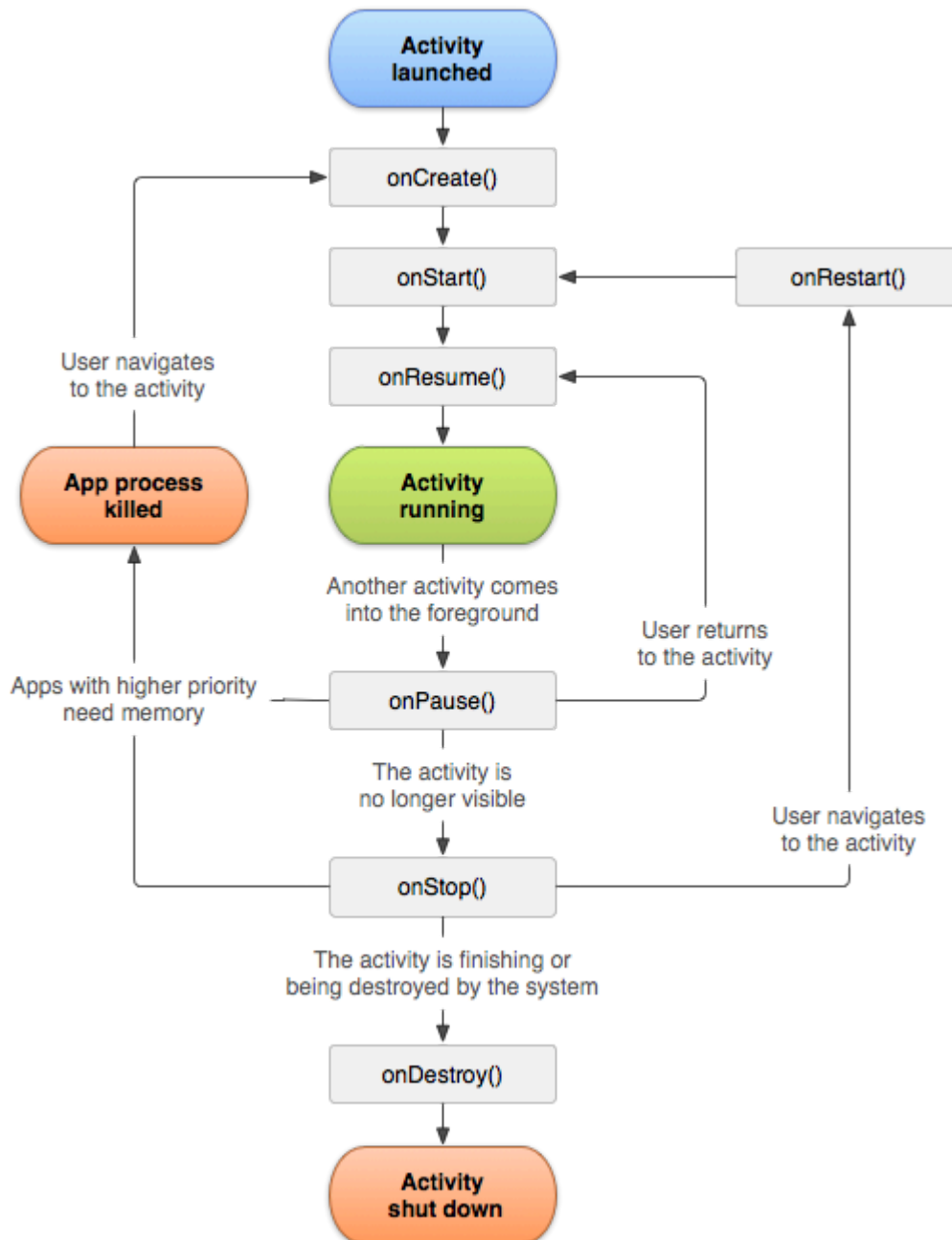
- The system calls `onStop()` when the activity is no longer visible to the user.
- This may happen because the activity is being destroyed or a new activity is starting and is covering the stopped activity.
- In all of these cases, the stopped activity is no longer visible at all.
- The next callback that the system calls is either `onRestart()`, if the activity is coming back to interact with the user, or by `onDestroy()` if this activity is completely terminating.

6. onRestart()

- The system invokes this callback when an activity in the Stopped state is about to restart. `onRestart()` restores the state of the activity from the time that it was stopped.
- This callback is always followed by `onStart()`.

7. onDestroy()

- The system invokes this callback before an activity is destroyed.
- This callback is the final one that the activity receives.
- `onDestroy()` is usually implemented to ensure that all of an activity's resources are released when the activity, or the process containing it, is destroyed.



Logcat

- The Logcat window in Android Studio helps you debug your app by displaying logs from your device in real time—for example, messages that you added to your app with the Log class, messages from services that run on Android, or system messages, such as when a garbage collection occurs.
- When an app throws an exception, Logcat shows a message followed by the associated stack trace containing links to the line of code.
- Each log has a date, timestamp, process and thread ID, tag, package name, priority, and message associated with it.
- Different tags have a unique color that helps identify the type of log.
- Each log entry has a priority of FATAL, ERROR, WARNING, INFO, DEBUG, or VERBOSE.
- we can Query logs using key-value search
- For example we can use

1. tag: Matches against the tag field of the log entry.

2. package: Matches against the package name of the logging app.

3. message: Matches against the message part of the log entry.
4. level: Matches the specified or higher severe log level—for example, DEBUG.

Log

- API for sending log output.
- Generally, we should use the Log.v(), Log.d(), Log.i(), Log.w(), and Log.e() methods to write logs.
- we can then view the logs in logcat.
- Log.d : Send a DEBUG log message.
- Log.e : Send an ERROR log message.
- Log.i : Send a INFO log message.
- Log.v : Send a VERBOSE log message.
- Log.w : Send a WARN log message.